Exp: 10

Aim: Image Denoising using Autoencoders

1. Dataset Source

Dataset used: MNIST Handwritten Digits Dataset

Source link:
https://www.kaggle.com/datasets/hojjatk/mnist-dataset

2. Dataset Description

- **Dataset Name:** MNIST
- **Type:** Image dataset
- **Total Samples:**
  - Training: 60,000 images
  - Testing: 10,000 images
- **Image Size:** 28 × 28 pixels
- **Color:** Grayscale (1 channel)
- **Classes:** Digits from 0 to 9

Features

- Pixel values (0–255) of each image

Target Variable

- Not required (unsupervised learning)
- Model learns to reconstruct clean images

Special Characteristic

- Noise is artificially added to images for training the denoising model

3. Mathematical Formulation of Autoencoder

An autoencoder consists of two parts:

1. Encoder

Compresses input into latent representation:

$z=f(x)$z=f(x)

Where:

- xx = input image
- zz = encoded (compressed) representation

2. Decoder

Reconstructs image from encoded data:

x^=$g(z)$x^=g(z)

Where:

- x^x^ = reconstructed image

3. Objective Function (Loss)

Minimize reconstruction error:

$Loss=||x-x$^||2Loss=||x−x^||2

OR (used in code):

$Loss=-[x\log(x$^)+(1−$x)\log(1-x$^)]Loss=−[xlog(x^)+(1−x)log(1−x^)]

(Binary Cross-Entropy)

4. Denoising Concept

Instead of clean input:

$xnoisy = x + noise$xnoisy=x+noise

Model learns:

$xnoisy \rightarrow x$xnoisy→x

4. Algorithm Limitations

- Requires large data for good performance
- Cannot recover details if noise is too high
- May produce blurred images
- Not suitable for structured/tabular data
- Sensitive to architecture design

5. Methodology / Workflow

Step 1: Load Dataset

- Import MNIST dataset

Step 2: Data Preprocessing

- Normalize pixel values (0–1)
- Reshape images

Step 3: Add Noise

- Add random Gaussian noise

Step 4: Build Model

- Encoder: Convolution + Pooling
- Decoder: Upsampling + Convolution

Step 5: Train Model

- Input: Noisy image
- Output: Clean image

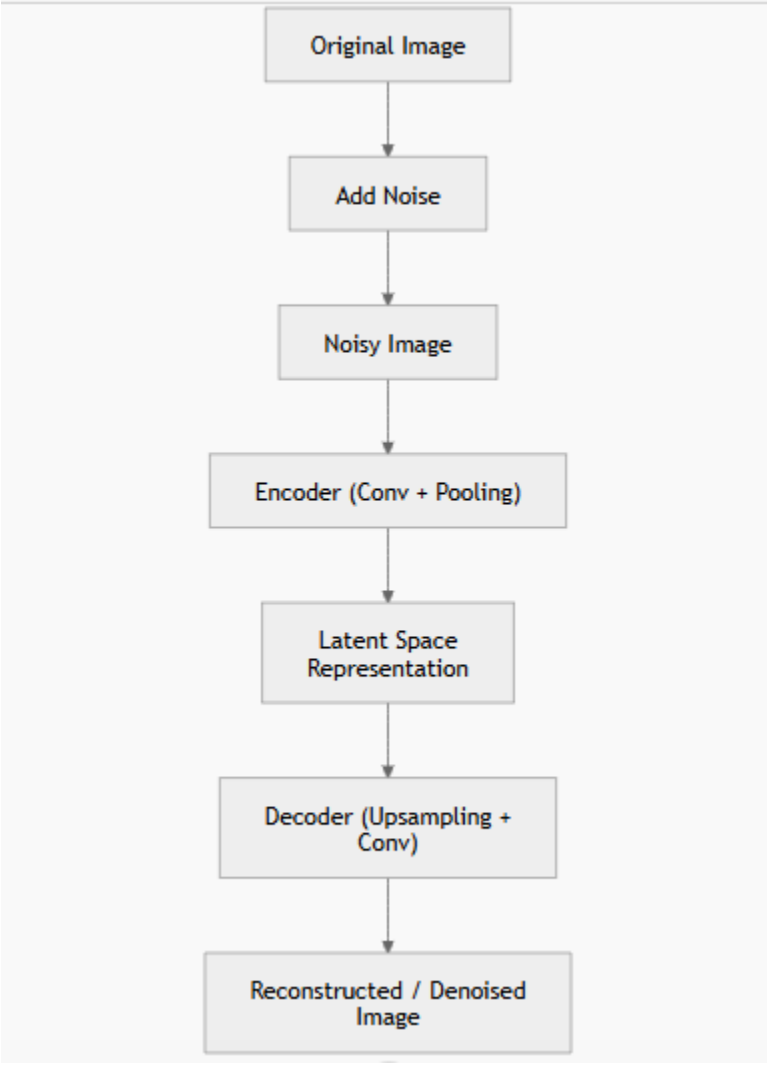
Step 6: Test Model

- Predict denoised images

Step 7: Visualization

- Compare:
    - Noisy image
    - Original image
    - Denoised image

Workflow Diagram (Text Form)



6. Performance Analysis

Evaluation Metrics

1. **Loss (Binary Crossentropy)**

$Loss=BCE(x,x$^)Loss=BCE(x,x^)

- Lower loss = better reconstruction

2. **Mean Squared Error (MSE)**

$MSE=1n\sum(x-x$^)2MSE=n1∑(x−x^)2

- Measures pixel difference

Observations

- Model reduces noise significantly
- Output images are smoother
- Some fine details may be lost

Result

- Denoised images closely resemble original images
- Model successfully learns noise removal

7. Hyperparameter Tuning

Parameters Tuned

Parameter	Values Tested
Epochs	5, 10, 20
Batch Size	64, 128
Learning Rate	0.001, 0.0005
Noise Factor	0.3, 0.5

Best Parameters

- Epochs = 10
- Batch Size = 128
- Learning Rate = 0.001
- Noise Factor = 0.5

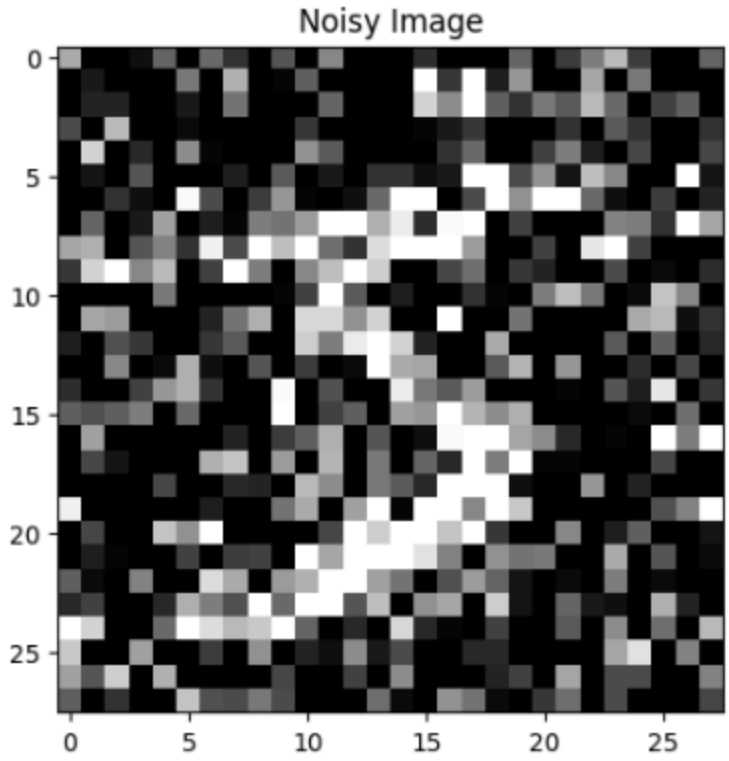
Impact of Tuning

- More epochs → better learning but risk of overfitting
- Higher noise → harder task but better generalization
- Proper batch size improves stability

Output

[6]
0s

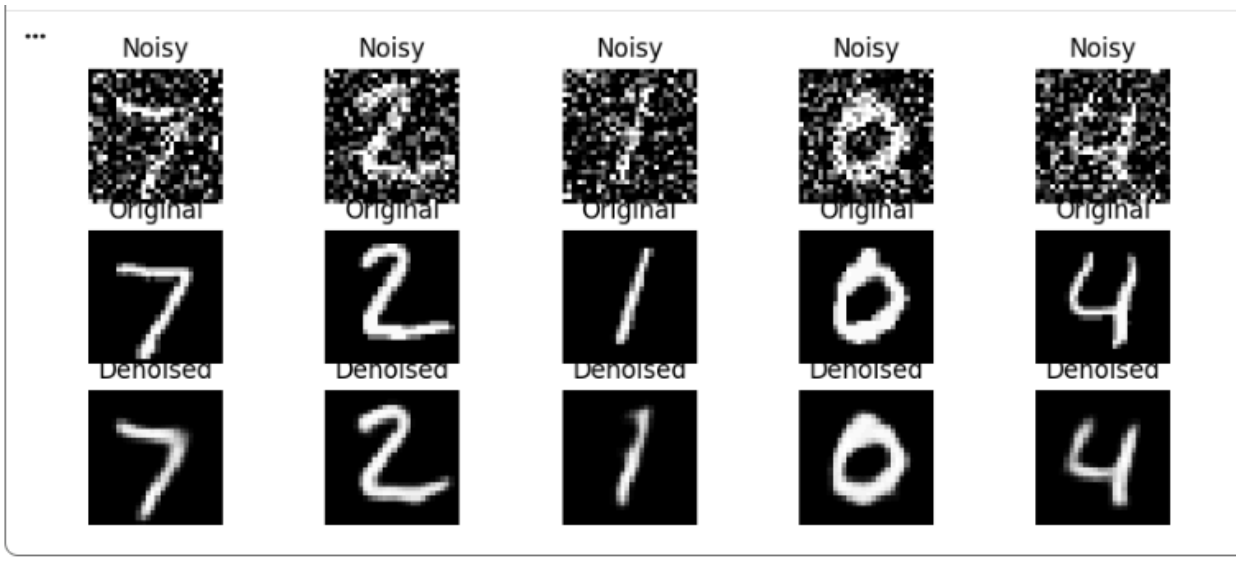
```
plt.imshow(x_train_noisy[0].reshape(28,28), cmap='gray')
plt.title("Noisy Image")
plt.show()
```



[9]
12m

```
history = autoencoder.fit(
    x_train_noisy, x_train,
    epochs=10,
    batch_size=128,
    shuffle=True,
    validation_data=(x_test_noisy, x_test)
)
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 68s 141ms/step - loss: 0.1732 - val_loss: 0.1205
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 78s 133ms/step - loss: 0.1164 - val_loss: 0.1121
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 64s 136ms/step - loss: 0.1108 - val_loss: 0.1079
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 82s 135ms/step - loss: 0.1077 - val_loss: 0.1055
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 79s 129ms/step - loss: 0.1058 - val_loss: 0.1043
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 83s 132ms/step - loss: 0.1044 - val_loss: 0.1026
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 59s 127ms/step - loss: 0.1032 - val_loss: 0.1016
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 81s 124ms/step - loss: 0.1022 - val_loss: 0.1016
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 61s 131ms/step - loss: 0.1015 - val_loss: 0.1001
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 85s 136ms/step - loss: 0.1009 - val_loss: 0.0999
```



Conclusion

- Autoencoders effectively remove noise from images
- Model learns compressed representation (latent space)
- Convolutional autoencoders perform better for images
- Useful in image processing and preprocessing tasks